Research Paper Classification

Complete Study Notes

Multinomial Logistic Regression from Scratch — Concepts, Math, Code & Evaluation

---

0. Big Picture: What This Notebook Does

> *This notebook builds a classifier that automatically labels a research paper as APPLIED, THEORETICAL, or SURVEY using only its title and abstract — with zero pre-built classifiers. Everything from math to code is done from scratch (except TF-IDF vectorization).*

The full pipeline in order:

- Load & inspect the dataset
- Preprocess text (clean → lowercase → remove stopwords → stem)
- Vectorize with TF-IDF (convert text to numbers)
- Build Multinomial Logistic Regression from scratch (no sklearn classifier)
- Train with gradient descent (300 epochs)
- Evaluate with confusion matrix, accuracy, precision, recall, F1
- Inspect confidence and misclassifications
- Save the model and run predictions on new papers

---

1. Libraries Used

Library	What It Does in This Project
numpy (np)	All matrix math — dot products, softmax, gradients, one-hot encoding
pandas (pd)	Loading CSV, displaying results, building summary tables
matplotlib / seaborn	All charts: bar plots, heatmaps, accuracy curves, term weights
sklearn.feature_extraction.text — TfidfVectorizer	Converts preprocessed text into TF-IDF feature matrices

Library	What It Does in This Project
sklearn.feature_extraction.text — ENGLISH_STOP_WORDS	Built-in set of common English words to remove (the, is, are...)
sklearn.model_selection — train_test_split	Splits data into train / validation / test sets with stratification
sklearn.metrics — accuracy_score, confusion_matrix, classification_report	Computes evaluation metrics after training
nltk.stem — PorterStemmer	Reduces words to their root form (running → run, models → model)
re (regex)	Cleans text using pattern matching (removes non-alphabet characters)
joblib	Saves and loads the trained model artifacts to/from disk
pathlib — Path	Clean file path management for saving model files
collections — Counter	Counts word frequencies for top-terms analysis

---

2. Dataset

File: data/combined_papers_cleaned.csv

Shape: 2,855 rows × 3 columns after dropping nulls

Column	Description
label	Class: APPLIED, THEORETICAL, or SURVEY
title	The paper's title
abstract	The paper's abstract text

Class Distribution

The dataset is perfectly balanced — each of the 3 classes has ~998 papers (~33% each). This is important because balanced classes mean we don't need to worry about the model ignoring minority classes.

Class	Count	% of Dataset
APPLIED	~998	~33%
THEORETICAL	~998	~33%
SURVEY	~998 (≈859 in test)	~33%

Visualization used: Bar chart with count labels using sns.barplot(), with counts and percentages annotated on top of each bar.

---

3. Text Preprocessing Pipeline

> *Goal: Turn raw, noisy text into clean, normalized tokens that carry maximum meaning for classification.*

3.1 Why Preprocess?

Raw text has noise: punctuation, capital letters, common words ('the', 'is', 'a'), and different word forms ('learns', 'learned', 'learning'). Preprocessing removes noise so the model learns from meaningful patterns.

3.2 The Four Steps

Step 1 — Lowercase

Converts all characters to lowercase. 'Neural' and 'neural' are the same word but different strings — lowercasing unifies them.

```
raw.lower()
```

Step 2 — Tokenization (regex)

Splits text into individual words using regex. The pattern [a-z]+ matches only lowercase alphabetic sequences — stripping numbers, punctuation, hyphens, etc.

```
re.findall(r'[a-z]+', text)
```

Example: 'Vision-Language-Action (VLA)' → ['vision', 'language', 'action', 'vla']

Step 3 — Stopword Removal

Removes common English words that appear in almost every document and carry no discriminative signal for classification. Uses sklearn's built-in ENGLISH_STOP_WORDS set.

```
[w for w in tokens if w not in stop_words]
```

Examples removed: the, is, are, this, for, with, on, in, a, an, of, to, be, it, its...

Step 4 — Porter Stemming

Reduces words to their root/stem form. This groups related words together so the model treats them as the same feature.

```
stemmer.stem(word)    # from nltk.stem.PorterStemmer
```

Original Word	Stemmed Form
learning	learn

Original Word	Stemmed Form
models	model
generalization	general
complexity	complex
networks	network
language	languag

Full Pipeline Function

```
def preprocess_text(t):
    tok = re.findall(r'[a-z]+', str(t).lower())
    no_stop = [w for w in tok if w not in stop_words]
    stemmed = [stemmer.stem(w) for w in no_stop]
    return ' '.join(stemmed)
```

Combining Title + Abstract

Title is short but information-dense. Abstract provides broader context. The two are concatenated before preprocessing:

```
df['combined_text'] = (df['title'] + ' ' +
df['abstract']).map(preprocess_text)
```

Visualization: Top Terms Before vs. After

A side-by-side horizontal bar chart (sns.barplot with orient='h') shows the 15 most frequent tokens before and after preprocessing. Before preprocessing, 'for', 'the', 'of' dominate — these are meaningless. After preprocessing, terms like 'model', 'learn', 'method', 'network' appear — these are meaningful for classification.

---

4. TF-IDF Vectorization

> *TF-IDF converts text into numbers. Each paper becomes a vector of numbers representing how important each word is to that paper relative to all other papers.*

4.1 The Three Formulas

Term Frequency (TF)

How often does this word appear in this document, relative to the document's total words?

TF(t, d) = (count of word t in document d) / (total words in document d)

If 'neural' appears 5 times in a document with 100 words: TF = 5/100 = 0.05

Inverse Document Frequency (IDF)

How rare is this word across all documents? Rare words are more informative.

IDF(t) = log(N / (1 + df(t))) + 1

Where: N = total number of documents, df(t) = number of documents containing word t

The '+1' in the denominator prevents division by zero. The outer '+1' ensures IDF is never zero.

A word in every document gets IDF ≈ 1 (low). A word in 1 document gets IDF ≈ log(N)+1 (high).

TF-IDF Score

TFIDF(t, d) = TF(t, d) × IDF(t)

A word scores high only if it appears frequently in this specific document AND rarely across all documents — that's what makes it discriminative.

4.2 sklearn TfidfVectorizer Settings

```
vectorizer = TfidfVectorizer(
    stop_words='english',    # Additional stopword removal
    max_features=12000,      # Keep only top 12,000 terms by frequency
    min_df=3,                # Ignore terms in fewer than 3 documents
    ngram_range=(1, 2)       # Use unigrams AND bigrams
)
```

What is an ngram?

A unigram is a single word: 'neural'. A bigram is a two-word phrase: 'neural network'. Bigrams capture phrase-level meaning that unigrams miss — 'deep learning' means something specific that 'deep' and 'learning' separately don't convey.

Sparsity

The TF-IDF matrix has shape (1827, 12000) — 1827 training papers × 12,000 features. Sparsity = 0.9917, meaning 99.17% of entries are zero. Each paper only uses a tiny fraction of the full vocabulary. This is why sparse matrix representation (scipy sparse) is used by sklearn.

Visualization: TF-IDF Heatmap

A 10×20 slice of the TF-IDF matrix is shown as a heatmap using sns.heatmap with cmap='mako'. Most cells are dark (near-zero), with occasional bright cells showing high-scoring terms. This visualizes sparsity directly.

4.3 No Data Leakage — The Key Rule

> *CRITICAL: vectorizer.fit_transform() is called ONLY on the training set. For validation and test sets, only .transform() is called. This prevents the model from 'seeing' test data during training.*

If you fit on all data, the vocabulary and IDF values would include knowledge of test documents — the model would artificially appear to perform better than it actually does.

```
vectorizer.fit_transform(X_train_text)    # Learn vocabulary + transform
vectorizer.transform(X_val_text)           # Only transform (no learning)
vectorizer.transform(X_test_text)          # Only transform (no learning)
```

4.4 Train / Validation / Test Split

Two-stage stratified split using sklearn's train_test_split with stratify=y to preserve class balance:

Split	Samples	Used For
Train	1,827 (64%)	Fit TF-IDF and train model weights
Validation	457 (16%)	Monitor overfitting during training
Test	571 (20%)	Final one-time evaluation after training

---

5. Multinomial Logistic Regression — From Scratch

> *Logistic regression is NOT built using sklearn's classifier. Every component — weights, forward pass, loss, gradients, update — is implemented manually using numpy.*

5.1 What Is a Logistic Regression Classifier?

A linear model that learns a score for each class, then converts scores to probabilities using softmax, and predicts the class with the highest probability.

For each paper, we compute: score = X · W + b

Where X is the TF-IDF vector, W is a learned weight matrix, and b is a learned bias vector.

5.2 Model Parameters

Weight Matrix W

Shape: (12000, 3) — one column per class, one row per TF-IDF feature.

Each column learns which words push toward that class. After training, high values in the SURVEY column might be at rows for words like 'survey', 'review', 'comprehensive'.

Initialized with small random values (0.01 × random normal) to break symmetry.

```
W = 0.01 * np.random.randn(12000, 3)
```

Bias Vector b

Shape: (1, 3) — one bias per class.

Allows the model to shift predictions even when all input features are zero.

```
b = np.zeros((1, 3))
```

5.3 Softmax — Converting Scores to Probabilities

Raw scores (called logits) can be any number — negative, zero, large positive. Softmax squashes them into probabilities that sum to 1.

Formula: softmax(z_i) = e^z_i / sum(e^z_j for all j)

Example: logits = [2.0, 1.0, 0.1] → softmax → [0.659, 0.242, 0.099]

All values are between 0 and 1, and they sum to exactly 1.0.

Numerical Stability Trick

Before computing e^z, we subtract the maximum logit from all logits. This prevents overflow (e^1000 = infinity) without changing the result mathematically:

```
z = logits - np.max(logits, axis=1, keepdims=True)
e = np.exp(z)
return e / e.sum(axis=1, keepdims=True)
```

5.4 Cross-Entropy Loss

Measures how wrong the model's predictions are. Lower is better.

Formula: L = -(1/N) * sum over all papers and classes: y_true * log(p_predicted)

Where y_true is 1 for the correct class and 0 for all others (one-hot encoded), and p_predicted is the softmax probability.

Why not Mean Squared Error (MSE)?

- MSE treats classification like regression — the distances between probability values don't have the right meaning
- Cross-entropy gives much stronger gradients when predictions are confidently wrong
- Paired with softmax, cross-entropy has a clean mathematical derivative: p - y_true

Concrete examples from the notebook:

```
cross_entropy([1,0,0], [0.90, 0.08, 0.02]) = 0.1054   # model mostly right → low loss
cross_entropy([1,0,0], [0.05, 0.90, 0.05]) = 2.9957   # model very wrong → high loss
```

Epsilon Clipping

We clip probabilities to [1e-12, 1-1e-12] to avoid log(0) = -infinity errors:

```
p = np.clip(p, 1e-12, 1 - 1e-12)
```

5.5 One-Hot Encoding

Labels (0, 1, 2) must be converted to one-hot vectors for the loss calculation:

```
APPLIED      = 0 → [1, 0, 0]
SURVEY       = 1 → [0, 1, 0]
THEORETICAL  = 2 → [0, 0, 1]

def one_hot(y, k):
    o = np.zeros((len(y), k))
    o[np.arange(len(y)), y] = 1
    return o
```

---

6. Training Loop — Gradient Descent

> *The training loop runs for 300 epochs. In each epoch, the model makes predictions, measures how wrong they are, computes which direction to adjust the weights, and takes a step in that direction.*

6.1 Hyperparameters

Hyperparameter	Value	What It Controls
epochs	300	How many times the model sees the full training set
learning_rate	0.5	Step size for weight updates — too large overshoots, too small learns slowly
l2 (regularization)	1e-4 (0.0001)	Penalty on large weights — prevents overfitting
print_every	20	How often to print progress during training

6.2 The Four Steps Per Epoch

Step 1 — Forward Pass (Compute Predictions)

```
logits = Xtr @ W + b          # Matrix multiply: (1827, 12000) @ (12000, 3) → (1827, 3)
p = softmax(logits)            # Convert scores to probabilities
loss = cross_entropy(Ytr, p)   # Measure how wrong we are
```

The @ operator is matrix multiplication in numpy. For each paper (row), we compute a score for each class (column) by taking the dot product of the paper's TF-IDF vector with each column of W.

Step 2 — Backward Pass (Compute Gradients)

Gradients tell us: 'if I increase this weight slightly, does the loss go up or down?'

```
dlog = (p - Ytr) / len(y_train)    # Gradient of loss w.r.t. logits
dW = Xtr.T @ dlog + l2 * W         # Gradient w.r.t. W (+ L2 regularization
term)
db = dlog.sum(axis=0, keepdims=True)  # Gradient w.r.t. b
```

The beautiful result: the gradient of cross-entropy loss with respect to the logits is simply (predicted_probability - true_label). This is one reason cross-entropy is preferred for classification.

Step 3 — Parameter Update

Move weights in the opposite direction of the gradient (to reduce loss):

```
W -= lr * dW
b -= lr * db
```

This is vanilla gradient descent. The learning rate controls how big each step is.

Step 4 — Monitor Progress

```
tr_pred = p.argmax(axis=1)                      # Pick highest-probability
class
val_pred = softmax(Xva @ W + b).argmax(axis=1) # Evaluate on validation set
train_acc = (tr_pred == y_train).mean()
val_acc   = (val_pred == y_val).mean()
```

6.3 L2 Regularization

L2 regularization adds a penalty term to the loss: (l2/2) * sum(W²)

This discourages large weight values, which helps prevent overfitting. The effect on the gradient is to add l2 * W to dW, which continuously pushes weights toward zero unless the training signal opposes it.

6.4 Training Progress

Epoch Range	Loss	Train Accuracy	Val Accuracy
0-1	1.0977	36.7%	42.2%
0-20	1.0484	68.3%	69.4%
40-60	0.9589	89.9%	89.3%
80-100	0.8812	93.9%	93.0%
140-160	0.7835	~95%	~95%
~300 (final)	~0.65	~96%	~96%

6.5 Training Curves Visualization

Two side-by-side plots (figsize 14x5):

- Left: Loss vs. Epoch — a smoothly decreasing curve. Loss starts near log(3)≈1.099 (random chance for 3 classes) and decreases as the model learns.
- Right: Accuracy vs. Epoch — two lines (train and validation) both rising, staying close together. Close tracking means the model is not overfitting.

---

7. Evaluation Metrics — Everything Explained

> *All evaluation is done on the held-out test set (571 samples) that the model has NEVER seen during training or hyperparameter tuning. The overall test accuracy is 95.97%.*

7.1 Confusion Matrix

A 3×3 grid where rows = true class, columns = predicted class. Diagonal cells = correct predictions. Off-diagonal cells = errors.

Visualization: sns.heatmap with annot=True, fmt='d', cmap='Blues'. Blue intensity represents the count.

How to read it: Look at row 'SURVEY'. If most numbers are on the diagonal (SURVEY predicted as SURVEY), the model classifies surveys well. If some shift to 'APPLIED', those are papers mistakenly labeled applied.

7.2 Precision, Recall, F1-Score

Metric	Formula	What It Means
Precision	TP / (TP + FP)	Of all papers predicted as class X, what fraction actually are class X?
Recall	TP / (TP + FN)	Of all actual class X papers, what fraction did we correctly identify?
F1-Score	2 × P × R / (P + R)	Harmonic mean of precision and recall — balanced single metric
Accuracy	(All correct) / (Total)	Overall fraction of correctly classified papers

TP = True Positive (correctly predicted this class), FP = False Positive (wrongly predicted this class), FN = False Negative (missed this class).

Why F1 and Not Just Accuracy?

If classes were imbalanced (e.g., 90% APPLIED), a model predicting 'APPLIED' for everything would get 90% accuracy but be useless. F1 catches this by also penalizing low recall. Since our dataset is balanced, accuracy ≈ macro F1 here, but F1 per class gives more diagnostic detail.

Macro vs. Weighted Averages

- Macro average: Average the metric across classes with equal weight per class (good for balanced datasets)
- Weighted average: Weight each class by its number of samples (better for imbalanced datasets)

7.3 Confidence Distribution

After softmax, each prediction has a max probability (the confidence). We plot a histogram (sns.histplot with kde=True) of all confidence values.

For a well-trained model, most confidences cluster near 1.0 — the model knows what it thinks. A uniform distribution would suggest the model is uncertain about everything.

7.4 Calibration Check

Calibration asks: 'When the model says it's 90% confident, is it right 90% of the time?'

We bucket predictions by confidence range and compare average accuracy in each bucket:

Confidence Bucket	Expected Accuracy	Good Model Behavior
0.0 – 0.4	Low	These uncertain predictions are often wrong
0.4 – 0.6	Medium	Roughly right half the time
0.6 – 0.8	High	Mostly correct
0.8 – 1.0	Very High	Almost always correct

7.5 Misclassification Review

We extract all incorrectly classified test papers (y_pred != y_test) and display their title, true label, predicted label, confidence, and a 350-character abstract snippet.

This is diagnostic — you can read actual misclassified papers to understand WHY the model fails. Common failure mode: survey papers that are also applied (dual-category papers) or theoretical papers about applied topics.

---

8. Interpreting the Model — Top Weighted Terms

Because logistic regression is a linear model, each feature (TF-IDF term) has an explicit weight for each class stored in W.

For each class, we sort the weights in W[:, class_index] from highest to lowest and plot the top 12 terms. These are the words that most strongly push the model toward predicting that class.

Visualization: One horizontal bar chart per class (sns.barplot with orient='h', color='#a5d6a7'). Bars pointing right = positive influence toward that class.

Typical patterns:

- SURVEY: terms like 'survey', 'review', 'overview', 'comprehens', 'recent'
- THEORETICAL: terms like 'proof', 'theorem', 'bound', 'complex', 'lower bound'
- APPLIED: terms like 'model', 'perform', 'result', 'achiev', 'benchmark', 'dataset'

---

9. Model Saving and Loading

> *joblib is used to serialize Python objects to disk. The model is saved as a single dictionary containing all artifacts needed for future inference — no retraining required.*

9.1 What Gets Saved

Key in Dict	Type	Purpose
vectorizer	TfidfVectorizer object	Same vectorizer used for training — needed to process new text consistently
weights	numpy array (12000, 3)	The learned W matrix — core of the classifier
bias	numpy array (1, 3)	The learned b vector
label_names	list of str	['APPLIED', 'SURVEY', 'THEORETICAL'] — class names in order
label_to_index	dict str→int	Converts class name to integer index
index_to_label	dict int→str	Converts integer index back to class name
preprocess_fn_name	str	Name of preprocessing function — reminder to apply same preprocessing

9.2 Save/Load Code

```
import joblib
from pathlib import Path

# Save
save_path = Path('models/tfidf_scratch/scratch_logreg_artifacts.joblib')
save_path.parent.mkdir(parents=True, exist_ok=True)
joblib.dump(artifact_dict, save_path)

# Load
loaded = joblib.load(save_path)
W_loaded = loaded['weights']
vectorizer_loaded = loaded['vectorizer']
```

9.3 Why joblib Instead of pickle?

joblib is optimized for numpy arrays and large objects. It compresses efficiently and is faster for scientific computing objects. It is the standard choice in the sklearn ecosystem.

---

10. Running Predictions on New Papers

The full inference pipeline for a new paper:

- Concatenate title + abstract
- Apply preprocess_text() — same function as training
- Transform with vectorizer.transform() — NOT fit_transform
- Compute logits: new_x @ W + b
- Apply softmax to get probabilities
- argmax to get predicted class index
- Map index back to class name using i2lab

```
new_text = preprocess_text(new_title + ' ' + new_abstract)
new_x = vectorizer.transform([new_text]).toarray()
probs = softmax(new_x @ W + b)[0]
pred_class = i2lab[int(np.argmax(probs))]
confidence = float(probs.max())
```

> *Demo result: 'A Survey of Efficient Transformer Compression Methods' was predicted APPLIED with 37.8% confidence (SURVEY had 37.1%). This was a borderline case — the paper is actually a survey, showing a real failure mode where closely named classes are ambiguous.*

---

11. Complete Math Reference

All Key Formulas in One Place

Concept	Formula / Rule
TF	count(word in doc) / total words in doc
IDF	log(N / (1 + doc_freq)) + 1
TF-IDF	TF × IDF
Linear score (logits)	Z = X · W + b, shape: (N, K)
Softmax	P_ik = exp(Z_ik) / sum_j(exp(Z_ij))
Numeric softmax	Z' = Z - max(Z per row), then same formula

Concept	Formula / Rule
Cross-entropy loss	L = -(1/N) * sum_n sum_k y_nk * log(P_nk)
Loss with L2	L_total = L_CE + (l2/2) * sum(W²)
Gradient: dL/dZ	(P - Y) / N, shape: (N, K)
Gradient: dL/dW	X^T · (dL/dZ) + l2 · W, shape: (F, K)
Gradient: dL/db	sum over rows of (dL/dZ), shape: (1, K)
Weight update	W = W - lr * dW
Bias update	b = b - lr * db
Prediction	argmax(softmax(X · W + b)) per row

Variable Key

Symbol	Shape	Meaning
X	(N, F)	TF-IDF matrix: N papers × F features (12,000)
W	(F, K)	Weight matrix: F features × K classes (3)
b	(1, K)	Bias vector: one per class
Z	(N, K)	Logits (raw scores before softmax)
P	(N, K)	Predicted probabilities from softmax
Y	(N, K)	One-hot true labels
N	scalar	Number of training samples (1,827)
F	scalar	Number of features (12,000)
K	scalar	Number of classes (3)
lr	scalar	Learning rate (0.5)
l2	scalar	L2 regularization strength (0.0001)

---

12. Anticipated Questions & Answers

Q: Why not use sklearn's LogisticRegression?

The point is to understand what happens inside. sklearn's classifier is a black box. By implementing it from scratch with numpy, every computation is visible and controllable.

Q: Why Porter Stemmer and not Lemmatization?

Porter stemmer is simpler and faster. Lemmatization requires POS tagging and a dictionary lookup (like in spaCy/NLTK WordNet), which is slower. For TF-IDF classification, stemming is usually good enough because the model learns from pattern matching, not semantics.

Q: Why max_features=12000?

A vocabulary cap limits memory and computation. Very rare words (in fewer than 3 documents due to min_df=3) are removed, and the top 12,000 by frequency are kept. This keeps the model tractable while retaining enough discriminative vocabulary.

Q: Why does validation accuracy match training accuracy so closely?

L2 regularization prevents overfitting. The model is also not particularly complex (linear, no deep layers), which makes generalization easier.

Q: What does 'balanced dataset' mean for evaluation?

Each class has equal samples. This means accuracy is a fair metric — there is no class-size bias. It also means the model can't cheat by always predicting the majority class.

Q: Why does the training loss not reach 0?

L2 regularization keeps weights from becoming too large, which adds a floor to the loss. Also, some papers are genuinely ambiguous between classes (applied-survey overlap), meaning even a perfect model would have some loss.

Q: What does the confidence histogram tell us?

If most predictions cluster near confidence=1.0, the model is decisive. If the distribution is flat, the model is uncertain. A well-calibrated model has high confidence on correct predictions and lower confidence on errors.

---

13. End-to-End Summary Table

Step	What Happens	Key Tool
1. Load	Read CSV, select label/title/abstract columns, drop nulls	pandas
2. Visualize Distribution	Bar chart of class counts to check balance	seaborn
3. Preprocess	lowercase → regex tokenize → stopword remove → Porter stem	re, nltk, sklearn
4. Combine Features	title + abstract → single preprocessed string	pandas
5. Split Data	80% train+val, 20% test (stratified). Then 80/20 train/val split.	sklearn
6. TF-IDF	Fit vectorizer on train only. Transform all 3 splits.	sklearn TfidfVectorizer

Step	What Happens	Key Tool
7. Initialize Model	W ~ N(0, 0.01), shape (12000,3). b = zeros (1,3).	numpy
8. Train	300 epochs: forward → loss → gradients → update. Track val acc.	numpy (from scratch)
9. Plot Curves	Loss curve + train/val accuracy curves over epochs	matplotlib
10. Top Terms	Plot highest W values per class to interpret model	matplotlib/seaborn
11. Evaluate	Test accuracy: 95.97%. Confusion matrix. Precision/Recall/F1.	sklearn metrics
12. Confidence	Histogram of max softmax prob. Calibration table by bucket.	numpy, pandas
13. Errors	Show misclassified papers with true/pred labels and confidence	pandas
14. Save	Save vectorizer + W + b + label maps as .joblib file	joblib
15. Predict	Load artifacts. Preprocess → transform → W·x+b → softmax → argmax	numpy

Final Test Accuracy: 95.97%

Trained from scratch · No sklearn classifier · Pure numpy gradient descent